

Comparative Analysis of Bipartite Graph Algorithms for Efficient Kidney Transplant Allocations

Advaith Subramanian Sahasranamam
Rutgers – New Brunswick

TABLE OF CONTENTS

Introduction	3
Experimental setup	4
Source code	6
Algorithms	6
Experimental results	9
Conclusion	17
Limitations and future work	18
Acknowledgements	19
References	19
Disclosure of AI assistance	20
Appendix	21

INTRODUCTION

Kidney failure is a serious medical condition that proves fatal without treatment. While kidney transplant is a life-saving treatment, the demand for donors currently outweighs their availability [1]. The challenge is not only about finding donors but also matching recipients to compatible donors under strict medical constraints. Rejection of a kidney by the recipient's immune system can have fatal consequences, with very acute cases resulting in death shortly after, or even during surgery [2]. An inefficient system can cost lives – donors can go unused while compatible recipients do not get matched.

Matching kidney donors to recipients can be represented as a bipartite graph matching problem [3]. This empirical study represents the problem as a bipartite graph $G = (V, E)$. Every vertex represents a donor/recipient, while an edge represents a match of donors and recipients. Current organ allocation systems must satisfy strict medical compatibility to be considered valid and reduce the risk of rejection. In the context of kidney transplantation, compatibility is determined by four factors [4]: ABO blood type, Rhesus (Rh) factor, HLA tissue typing across three loci (HLA-A, HLA-B, HLA-DR), and the recipient's Panel Reactive Antibody (PRA). Only pairs satisfying these constraints are added to an edge in the graph.

Current allocation systems, such as those used by the United Network for Organ Sharing (UNOS), use a points-based system that considers factors such as HLA compatibility, time on the waiting list, and recipient sensitization [5]. While practical, this approach is a greedy heuristic that does not provide a globally optimal solution – matching is inefficient, which leaves behind compatible donor-recipient pairs due to suboptimal decisions. Research shows that applying a more efficient algorithm to the donor pool could increase the number of successful transplants by 5-10% annually [6].

In pursuit of this goal, this work presents an empirical analysis of three algorithms of increasing complexity. A greedy matching algorithm serves as the baseline, mirroring the heuristic nature of current allocation systems. Kuhn's algorithm [7] builds upon this by implementing augmenting path search, finding the maximum number of compatible matches. The Jonker-Volgenant version of the Hungarian algorithm [8] optimizes matches by minimizing the transplantation risk through shortest path augmentation and dual variable maintenance, producing matches of higher medical quality – leading to lower risks of rejection.

EXPERIMENTAL SETUP

Data generation:

The algorithms are evaluated on randomly generated bipartite with graph size (n) varying from $n=100$ to $n = 700$ donor-recipient pairs. The data was synthetically generated using Python's *random* library, with a seed being set for reproducibility and statistical value.

ABO Blood type is a hard constraint in kidney donor-matching, and an incompatible blood type triggers an immediate immune rejection response. Blood type data are generated and assigned using a weighted distribution after the U.S./European population (Table 1). Compatibility follows donation rules. Blood type O is a universal donor, AB a universal acceptor, and other blood groups donating and receiving solely from their own groups. [9]

The Rhesus (Rh) factor is a secondary blood group antigen considered in kidney transplantation. Rh- donors are universal donors, while Rh+ donors can only donate to other Rh+ recipients. Rh factors are also modeled after the U.S./European population in the year 2024 [8] (Table 2).

HLA tissue typing is the primary determinant of long-term graft survival in kidney transplantation. Each person inherits two alleles per locus across three loci. This gives six total alleles per person, and compatibility is measured as the number of shared alleles between donor and recipient on a scale of 0 to 6, where 6 represents a perfect match. Unlike the other distributions, however, HLA allele frequencies are complex and highly population specific. To this end, alleles are sampled uniformly without replacement or homozygosity (two of the same alleles per locus) to simplify the model – noted as a limitation of this model.

Panel Reactive Antibody (PRA) score measures the degree to which a recipient has developed antibodies against potential donors, expressed as a percentage from 0 to 100. A higher PRA indicates greater sensitization – a higher risk of the recipient's immune system rejecting the donor's kidney. PRA values are generated using a three-tier mixture distribution to reflect sensitization levels (Table 3). The exact distribution varies between transplant centres and year – hence, this model uses an approximation of the PRA values.

Blood Type	Distrubution
O	44%
A	42%
B	10%
AB	4%

Table 1: Blood Type Distrubution

Rhesus Factor	Distribution
Rh+	85%
Rh-	15%

Table 2: Rh Factor Distribution

Locus	Number of Alleles	Distribution
HLA-A	24	Uniform
HLA-B	52	Uniform
HLA-DR	18	Uniform

Table 3: HLA Allele Distribution

Recipient Population	PRA value distribution
70% of recipients – Low PRA	Beta(1, 8) – skewed toward 0
20% of recipients – Moderate PRA	Beta(2, 2) – centered around 50
10% of recipients – High PRA	Beta(8, 1.5) – skewed toward 100

Table 4: PRA Distribution

Compatibility Constraints

A donor-recipient pair is considered compatible and forms an edge of the bipartite graph only if the following four constraints are satisfied: First, ABO blood type compatibility must hold following standard donation rules. Second, Rh factor compatibility must hold. Third, a virtual crossmatch test is performed using the recipient’s PRA score and the donor-recipient HLA match score. The crossmatch rejection probability is computed as:

$$\text{rejection probability} = \frac{\text{PRA}}{100} * \left(1 - \frac{\text{HLA Score}}{6}\right)$$

A pair is compatible if the rejection probability falls below a specific threshold parameter *rej_chance*. This threshold is set to 0.2 in all experiments, except when analyzing the effects of *rej_chance* on the algorithm’s performance. An HLA score of 6/6 results in zero rejection risk regardless of PRA, while a 0/6 results in certain rejection. It is important to note that, in the real world, even a low PRA and a high HLA score may sometimes result in rejection. Hence, this is a limitation of the current model.

Constraint	Rule
Blood Type	Donor type must be compatible with recipient.
Rh	Rh+ cannot donate to Rh-
HLA	Counted across the alleles (Score of 0/6)
Crossmatch	Rejection probability < <i>rej_chance</i> threshold

Table 5: Compatibility rules

Experiments run

Three sets of experiments were conducted. The first evaluates scalability by varying the graph size – the number of donors and recipients (n) are both equal, which n ranging from 100 to 500 in steps of 5, giving 81 graph sizes in total. Each size is evaluated over 3 independent trials, with the results being averaged. This balanced configuration models a scenario where a donor and recipient pool are equal, and also as a baseline measure of performance.

The second experiment evaluates the effect of the donor-recipient ratio performance. The number of recipients is fixed at 500 while the donor-recipient ratio is varied from 0.1 to 0.9 in steps of 0.1 – giving donor pool sizes ranging from 50 to 450. This models real-world scenarios where recipients outnumber donors. Each ratio is evaluated over 5 independent trials and the results are averaged.

The third experiment evaluates the effect of the rejection probability threshold. The number of donors and recipients are fixed at 500, while the rejection threshold is varied from 0.05 to 0.5 in steps of 0.05. This models a scenario where some applications would may varying thresholds of rejection.

Computing specifications:

These experiments were completed using Python 3.14, using VSCode IDE. A MacBook Air, with an M2 processor and 16GB of RAM was used, running MacOS Tahoe 26.1

SOURCE CODE:

The source code for this project, with all relevant instructions, can be found in the author's GitHub repository at: <https://github.com/AdSahas/kidney-bipartite-matching>

ALGORITHMS EMPLOYED

Greedy Matching

Greedy matching is the simplest approach to bipartite matching, serving as the baseline in this study. The algorithm iterates across donors in order and assigns them to the first compatible and available recipient. A match which has been made is never reconsidered, and there exists no backtracking or augmentation. This mirrors the heuristic nature of current organ allocation systems such as UNOS, which assign donors greedily based on a scoring function.

The asymptotic time complexity of the greedy algorithm is $O(D \cdot R)$, where D represents the number of donors, and R the number of recipients. In the worst case, the algorithm checks all possible recipients for each donor. In the experiments, however, this worst case did not occur. While faster in complexity, this method is suboptimal in terms of maximal matching and quality of matches – a locally good decision can block a globally optimal matching.

Algorithm: Greedy Kidney Transplant Matching

Input: Donor set D , Recipient set R , rejection threshold τ

Output: Matching M

```
1   Initialize  $M[r] \leftarrow \text{NIL}$  for all  $r \in R$ 
2   for each donor  $d \in D$  do
3       for each recipient  $r \in R$  do
4           if  $M[r] = \text{NIL}$  and  $\text{COMPATIBLE}(d, r)$  and  $\text{CROSSMATCH}(d, r) < \tau$  then
5                $M[r] \leftarrow d$ 
```

```

6         break
7     return M

```

Figure 1: Greedy Algorithm Pseudocode

Kuhn's Algorithm

Kuhn's algorithm finds the maximum bipartite matching in a graph through augmenting path search. Unlike the greedy algorithm, Kuhn's algorithm reassigns existing matches to free space for donors that would otherwise be unmatched. This is accomplished using a depth first search (DFS) that, for each donor, attempts to find an augmenting path. The key insight of this algorithm is the recursive reassignment of matched pairs. When a recipient is already matched, Kuhn's asks whether their current donor can be reassigned to a different recipient. This chain of reassignments is what allows Kuhn's to create matches that greedy otherwise misses. A visited array is maintained per donor iteration to prevent cycles during the DFS.

The algorithm has an asymptotic time complexity of $O(V \cdot E)$, with V representing the number of vertices and E the number of edges. While asymptotically equivalent to the greedy method, Kuhn's often performs better due to its backtracking capabilities.

Algorithm: Kuhn's Kidney Transplant Matching**Input:** Donor set D , Recipient set R , rejection threshold τ **Output:** Matching M

```

1  Initialize  $M[r] \leftarrow \text{NIL}$  for all  $r \in R$ 
2  for each donor  $d \in D$  do
3      visited[r]  $\leftarrow$  False for all  $r \in R$ 
4      tryAugment( $d$ , visited,  $M$ )
5  return  $M$ 

process tryAugment( $d$ , visited,  $M$ ):
6      for each recipient  $r \in R$  do
7          if visited[r] = False and COMPATIBLE( $d$ ,  $r$ ) and CROSSMATCH( $d$ ,  $r$ )  $< \tau$  then
8              visited[r]  $\leftarrow$  True
9              if  $M[r] = \text{NIL}$  then
10                  $M[r] \leftarrow d$ 
11                 return True
12             if tryAugment( $M[r]$ , visited,  $M$ ) then
13                  $M[r] \leftarrow d$ 
14                 return True
15     return False

```

Figure 2: Kuhn's Algorithm Pseudocode

Jonker-Volgenant (JV) Algorithm

The Jonker-Volgenant algorithm solves the linear assignment problem - finding the minimum cost perfect matching in a weighted bipartite graph. Unlike Kuhn's algorithm, which maximizes the number of matches without considering the quality, Jonker-Volgenant minimizes the total assignment cost across all matched pairs. In this context, the total assignment cost refers to assignments that minimize total rejection risk. This produces fewer matches, but with higher quality – corresponding to matches of better compatibility than both Kuhn's and Greedy algorithms. This algorithm maintains two sets of dual variables: donor potentials u and recipient potentials v – that together define reduced costs for each donor-recipient pair:

$$reduced\ cost(d, r) = cost(d, r) - u[d] - v[r]$$

This was also the most challenging to implement – with an understanding of Linear Programming serving as a baseline for understanding the update rules (See appendix for the linear programming concept that Jonker-Volgenant follows). As a variant of the Hungarian Algorithm, these potentials replace the matrix reduction step of the classical Hungarian algorithm, making Jonker-Volgenant significantly faster in practice. For each unmatched donor, Dijkstra's algorithm runs on the reduced cost graph to find the shortest augmenting path to a free recipient. After each augmentation, the potentials are updated to maintain non-negativity of reduced costs for future calls to the Dijkstra's algorithm. This guarantees that each subsequent shortest path search is valid and that the final assignment is globally optimal. The cost of a donor-recipient pair is defined as:

$$cost(d, r) = rejection\ chance(d, r) \times \left(1 - \frac{HLA\ Score(d, r)}{6}\right)$$

Incompatible pairs – those that do not satisfy the constraints mentioned in this study – are given a cost of infinity, so that they are never selected.

The asymptotic time complexity of Jonker-Volgenant, implemented in this study as a simple array-based Dijkstra, is $O(n^3)$. This matches with the Hungarian algorithm's time complexity, but the coefficients are smaller for Jonker-Volgenant than the classic Hungarian implementation. This is tied to the fact that the potential update mechanism mentioned above avoids the repeated matrix adjustment and line-cover steps of the Hungarian algorithm.

Algorithm: Jonker-Volgenant Kidney Transplant Matching

Input: Donor set D , Recipient set R , rejection threshold τ

Output: Matching M

- 1 Initialize $u[d] \leftarrow 0$ for all $d \in D$
 - 2 Initialize $v[r] \leftarrow 0$ for all $r \in R$
 - 3 Initialize $matchD[d] \leftarrow NIL$ for all $d \in D$
 - 4 Initialize $matchR[r] \leftarrow NIL$ for all $r \in R$
-

```

5   for each donor  $d \in D$  do
6     ( $r_{\text{Free}}$ ,  $\text{dist}$ ,  $\text{prevD}$ ,  $\text{prevR}$ )  $\leftarrow$  Dijkstra( $d$ ,  $u$ ,  $v$ ,  $\text{matchD}$ ,  $\text{matchR}$ ,  $\tau$ )
7     if  $r_{\text{Free}} = \text{NIL}$  then continue
8     update  $u[d']$  for all matched donors  $d'$ 
9     update  $v[r']$  for all recipients  $r'$ 
9     augment along path from  $d$  to  $r_{\text{Free}}$  using  $\text{prevD}$ ,  $\text{prevR}$ 
10    return  $\text{matchD}$ 

process Dijkstra( $d$ ,  $u$ ,  $v$ ,  $\text{matchD}$ ,  $\text{matchR}$ ,  $\tau$ ):
11    Initialize  $\text{dist}[r] \leftarrow \text{INF}$  for all  $r \in R$ 
12    Initialize  $\text{visited}[r] \leftarrow \text{False}$  for all  $r \in R$ 
13    Initialize  $\text{prevD}[r] \leftarrow \text{NIL}$  for all  $r \in R$ 
14    Initialize  $\text{prevR}[r] \leftarrow \text{NIL}$  for all  $r \in R$ 
15    for each recipient  $r \in R$  do
16       $\text{cost} \leftarrow \text{COST}(d, r, \tau)$ 
17      if  $\text{cost} < \text{INF}$  then  $\text{dist}[r] \leftarrow \text{cost} - u[d] - v[r]$ 
18    while True do
19       $r_{\text{Min}} \leftarrow \text{argmin}\{\text{dist}[r] : \text{visited}[r] = \text{False}\}$ 
20      if  $r_{\text{Min}} = \text{NIL}$  or  $\text{dist}[r_{\text{Min}}] = \text{INF}$  then return NIL
21       $\text{visited}[r_{\text{Min}}] \leftarrow \text{True}$ 
22      if  $\text{matchR}[r_{\text{Min}}] = \text{NIL}$  then return ( $r_{\text{Min}}$ ,  $\text{dist}$ ,  $\text{prevD}$ ,  $\text{prevR}$ )
23       $d_{\text{Next}} \leftarrow \text{matchR}[r_{\text{Min}}]$ 
24      for each recipient  $r \in R$  do
25        if  $\text{visited}[r] = \text{False}$  then
26           $\text{cost} \leftarrow \text{COST}(d_{\text{Next}}, r, \tau)$ 
27          if  $\text{cost} < \text{INF}$  then
28             $\text{newDist} \leftarrow \text{dist}[r_{\text{Min}}] + \text{cost} - u[d_{\text{Next}}] - v[r]$ 
29            if  $\text{newDist} < \text{dist}[r]$  then
30               $\text{dist}[r] \leftarrow \text{newDist}$ 
31               $\text{prevD}[r] \leftarrow d_{\text{Next}}$ 
32               $\text{prevR}[r] \leftarrow r_{\text{Min}}$ 

```

Figure 3: Jonker-Volgenant Algorithm Pseudocode

EXPERIMENTAL RESULTS

Experiments with equal donors and recipients

Runtime: Greedy matching demonstrates a near-constant runtime across all graph sizes, confirming the early termination effect. Once a compatible donor is found, the inner loop exits immediately, moving on to the next one. Kuhn's algorithm shows a larger time complexity, roughly $O(n^3)$ worst-case behaviour as backtracking and finding an augmenting path in larger graphs become increasingly expensive. Jonker-Volgenant is the slowest of all three algorithms, taking almost $O(n^3)$ worst-case time, with a larger coefficient than Kuhn's. The noisy growth rate of Jonker-Volgenant likely reflects the sensitivity of the algorithm to the randomly generated graphs. Graphs with varying distributions of compatible pairs can affect the time taken to find optimal augmenting paths through Dijkstra's algorithm. Increasing the number of trials per size

would also produce a smoother graph, but with the inability of quadratic functions to scale satisfactorily, it is infeasible without more powerful computing resources.

The trends were fit with a curve that matches their theoretical time complexity, showing relatively high R^2 values – confirming that the theoretical nature matches their actual performance almost exactly.

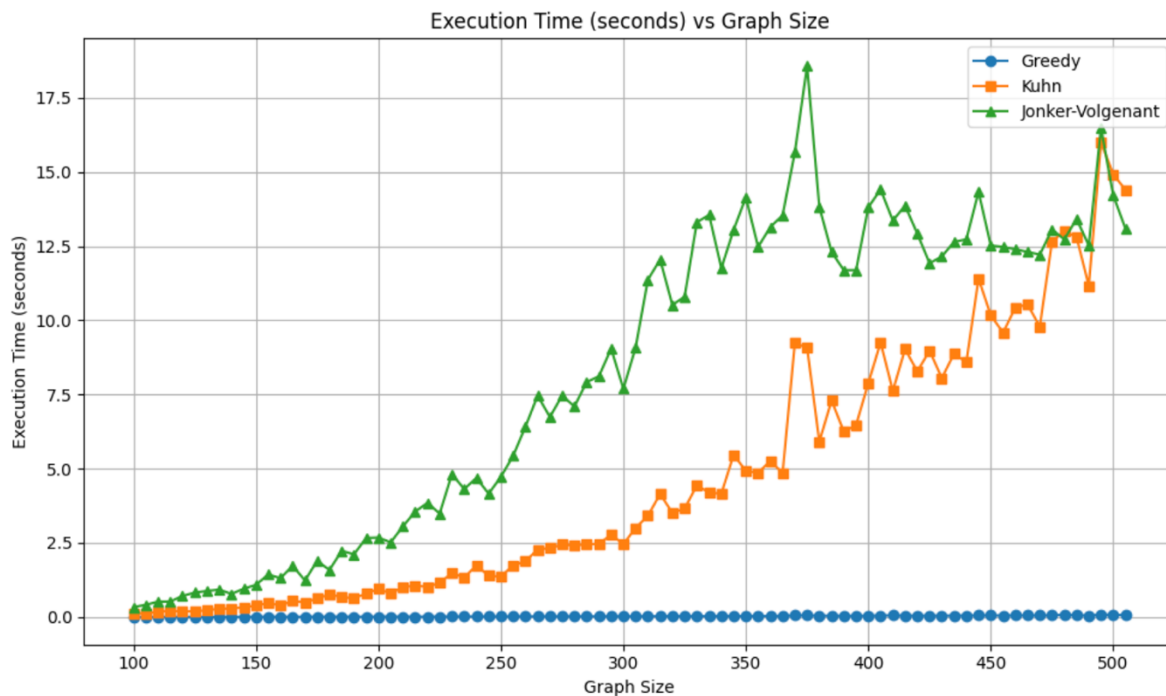


Figure 4: Effect of graph size on execution time

Algorithm	Theoretical Complexity	R^2 value (2 d.p)
Greedy Matching	$O(n^2)$	0.94
Kuhn's Algorithm	$O(n^2)$	0.97
Jonker-Volgenant Algorithm	$O(n^3)$	0.94

Table 5: Correlation analysis with theoretical curves

Compatible matches: All three algorithms show a consistent increase in the number of matches as problem size grows, reflecting a larger number of compatible pairings as more donors and recipients become available. Both Kuhn's and Greedy algorithms showed comparable numbers of matchings for relatively small graphs of $n < 350$, when Kuhn's starts to diverge above Greedy - as it starts to backtrack over more nodes to recover the number of matches, while Greedy's suboptimal choices are more detrimental. This result shows that for smaller graph sizes, Greedy does not significantly block the possibility of optimal matches. Jonker-Volgenant algorithm plateaus at $n = 350$, diverging significantly from others. This highlights the tradeoff that Jonker-Volgenant makes, by prioritizing quality over quantity.

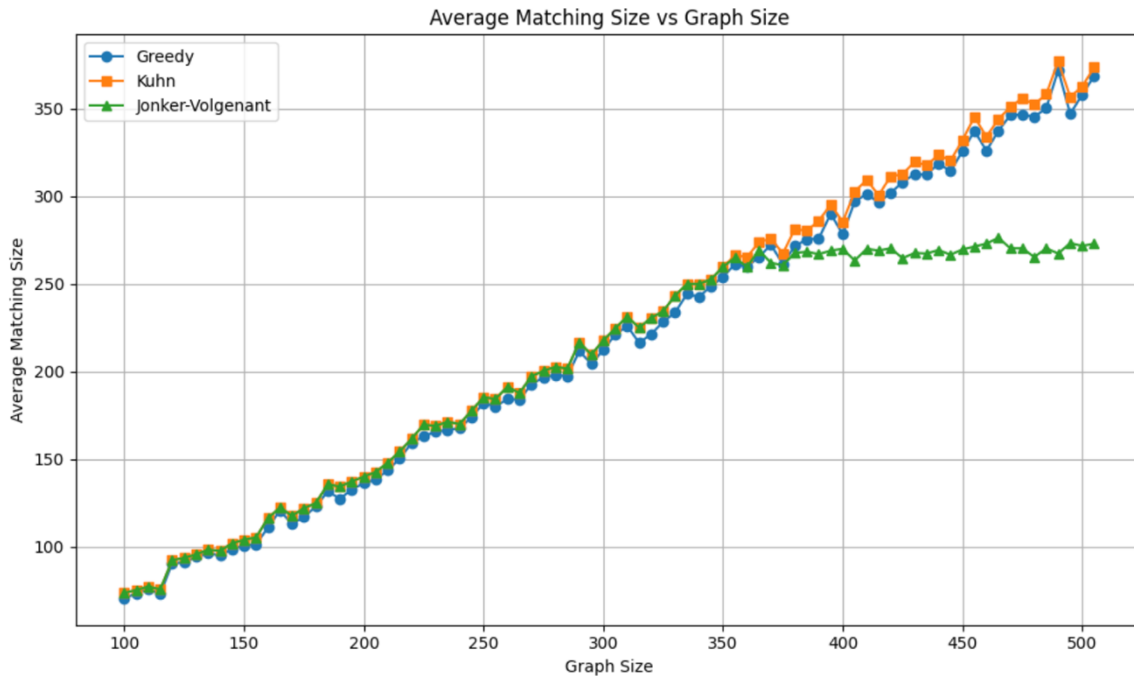


Figure 5: Effect of graph size on matching size

Match Quality: The average HLA metrics in the experiments reveal a clear separation between the algorithms. JV consistently produced the highest average in HLA metrics, with an upward trend from $n \sim 400$. This confirms its successful ability to perform high-quality matches. While Kuhn's produces a comparable HLA score for smaller graph sizes, it falls behind at larger n . Greedy produces significantly lower HLA scores – confirming the locally optimal decisions do not attempt to improve match quality at all.

PRA results reveal findings that were unexpected. Greedy matches recipients with dramatically higher average PRA compared to the others. This suggests that Greedy's locally optimal decisions unintentionally favour highly sensitized recipients, likely because they appeared earlier in the recipient iteration order. JV and Kuhn's produce consistently low PRA scores, although JV trends slightly higher for larger sizes. This matches the algorithm's cost function, as a high PRA recipient with a strong HLA match can still have a low rejection chance. JV will preferentially select such pairs to minimize total cost.

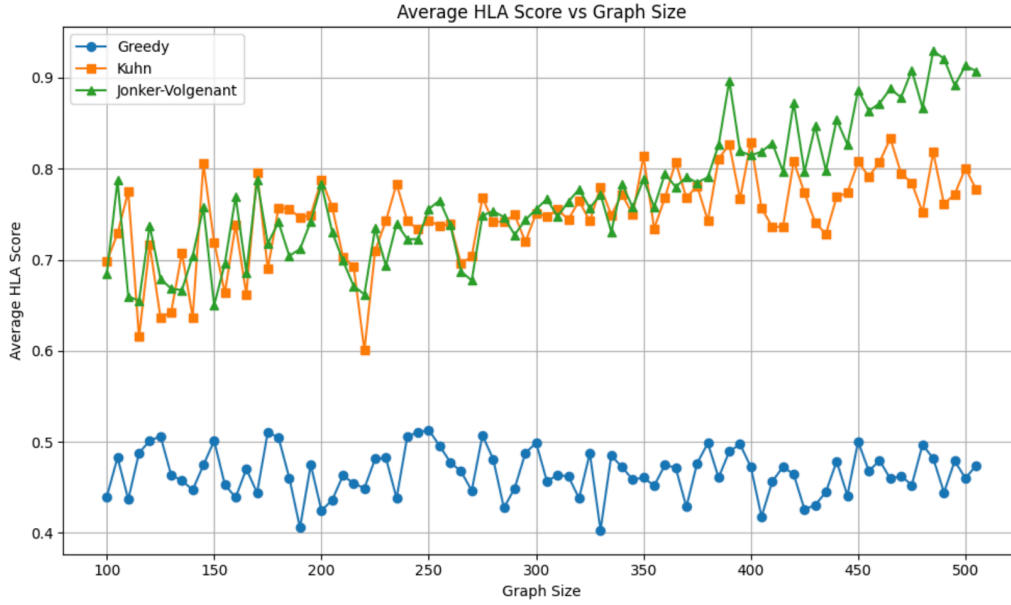


Figure 6: Effect of graph size on HLA scores

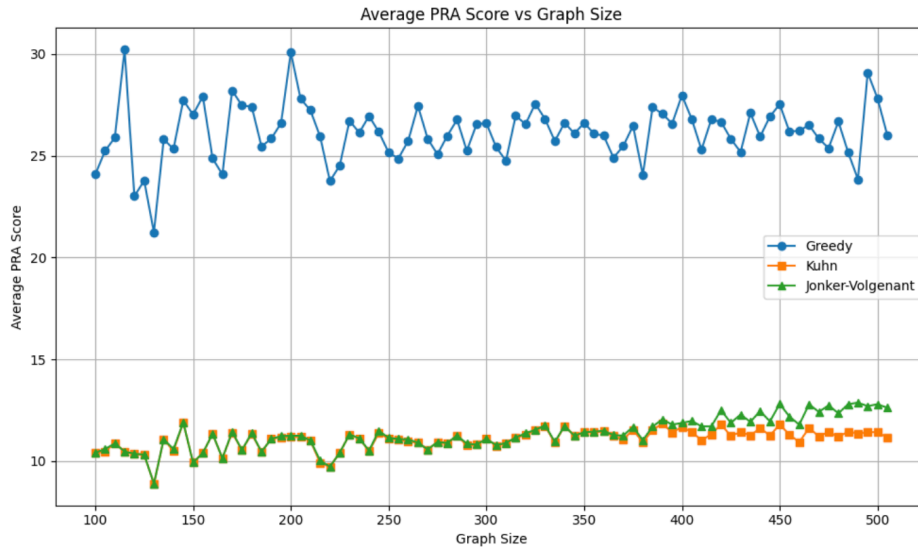


Figure 7: Effect of graph size on PRA scores

Experiments with varying ratio of donors to recipients

Runtime: Similar to the previous runs, the algorithms performed similarly for varying ratios in terms of time complexity. Greedy matching remained quadratic regardless of ratio, consistent with its $O(\alpha R^2)$ complexity and early termination. Kuhn's runtime also grew with increasing ratio, while its worst case was predicted as $O(\alpha^2 R^3)$. However, empirical growth remained closer to quadratic while testing, as short augmenting paths in sparser graphs prevent the cubic behaviour that was expected. Similarly, Jonker-Volgenant exhibited similar runtime as previously observed.

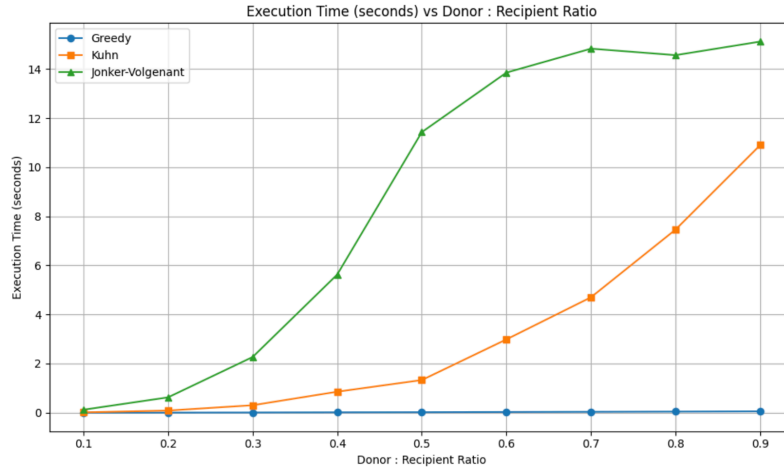


Figure 8: Effect of donor-recipient ratios on execution time

Algorithm	Theoretical Complexity	R ² value (2 d.p)
Greedy Matching	$O(n^2)$	0.99
Kuhn's Algorithm	$O(n^2)$	0.99
Jonker-Volgenant Algorithm	$O(n^3)$	0.98

Table 6: Correlation analysis with theoretical curves

Matching size: With this experiment, the matching sizes show some similarities with the previous experiment. At low donor-recipient ratios, all three algorithms show identical, if not very similar, matching sizes. The donor pool is small enough that every donor finds a compatible recipient regardless of algorithm. This confirms that when donors are scarce, the bottleneck is the availability, and not algorithm quality. Beyond ratio 0.5, the algorithms diverge. Greedy and Kuhn's algorithms continue to grow together up to ratio 0.6, after which Kuhn's pulls ahead. This is consistent with the fact that Kuhn's uses augmenting paths to re-assign donors and recipients. JV plateaus at approximately 270 matches, mirroring its behaviour from the previous experiments. The convergence of all three algorithms at low ratios is a clinically important finding. When donors are scarce, the choice of algorithm does not significantly affect the number of matches. Algorithm choices matter only when the sparsity of donors is not very large.

Match Quality: The quality metrics across donor-recipient ratios reveal an intriguing pattern for JV. At low ratios, JV produces dramatically higher HLA scores, before declining sharply as ratio increases. This can be explained by the scarcity of donors at low ratios – with very few donors, JV only selects the highest quality matches, leaving many unmatched. As the pool grows, it is skewed towards accepting lower-quality matches, while still showing a higher average HLA score than the other algorithms. Kuhn's and Greedy show the opposite behaviour, with their scores rising, but not reaching the quality of JV.

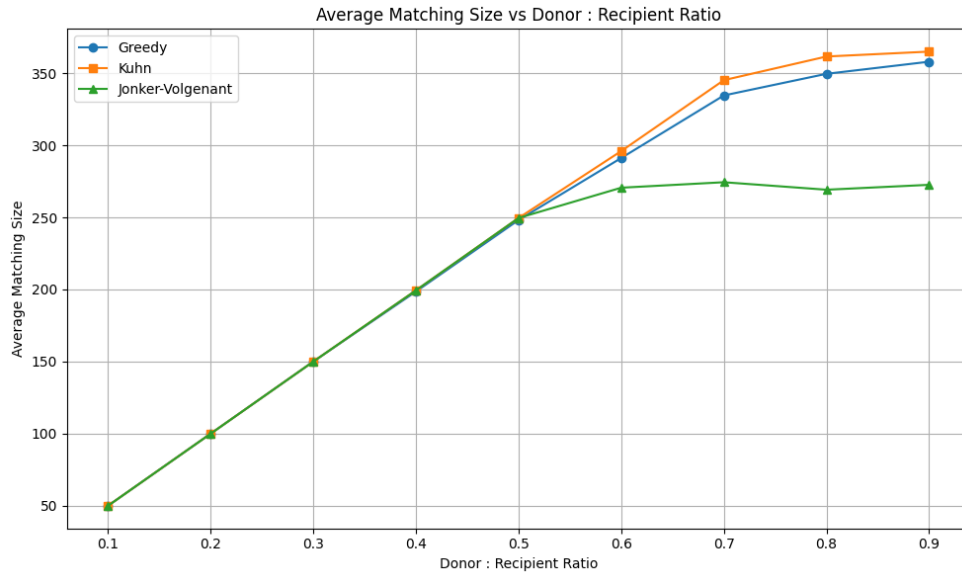


Figure 9: Effect of donor-recipient ratios on execution time

The PRA scores also reveal information in support of the interpretation. JV is initially forced into accepting higher PRA scores in the pool. As the graph gets more balanced, these scores drop – whereas, for Kuhn’s and Greedy, the trend is once again the opposite. The PRA values rise, showing that they, once again, do not account for match quality. However, JV shows higher overall PRA values for every ratio. This shows that JV, for cases where donors are sparser, may match more highly sensitized recipients.

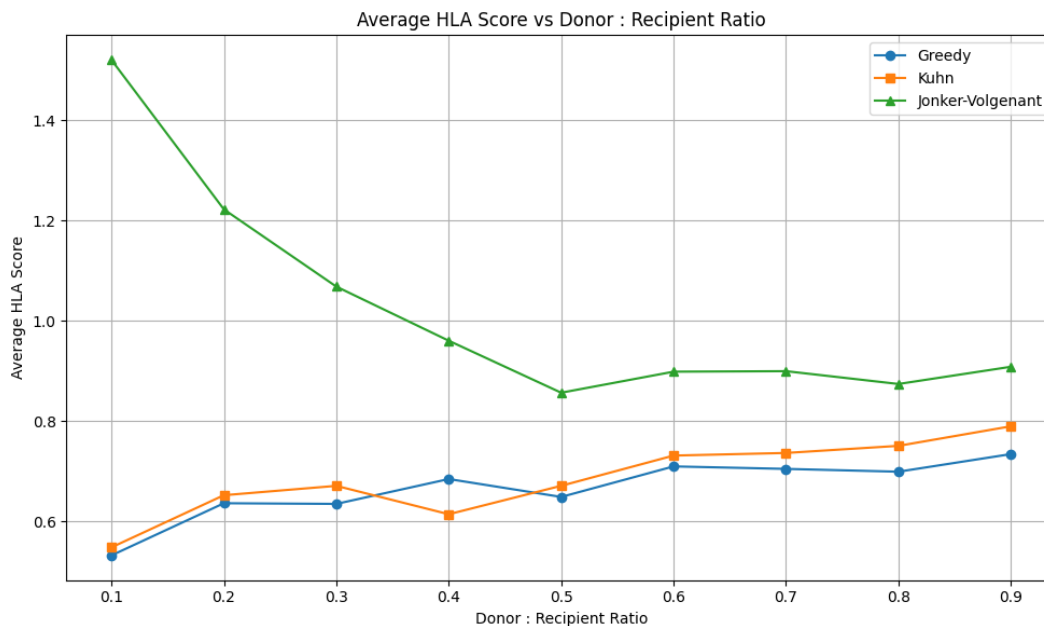


Figure 10: Effect of donor-recipient ratios on HLA



Figure 11: Effect of donor-recipient ratios on PRA

Experiments with varying rejection thresholds

Runtime: The runtime results reveal an important finding, which may be counterintuitive. As the rejection threshold increases, the runtime of the algorithms decreases, despite the graph becoming denser with more possible matches. While at low thresholds the graph is sparse, compatible pairs are rare – and Kuhn’s DFS must exhaustively search for long augmenting paths before finding free recipients. As the threshold increases, recipients become easier to match directly, with augmenting paths becoming shorter. JV exhibits an interesting peak at ratio of 0.1, highlighting the overhead that Dijkstra’s algorithm produces for the augmentation in sparse graphs – where many donors require long shortest path searches.

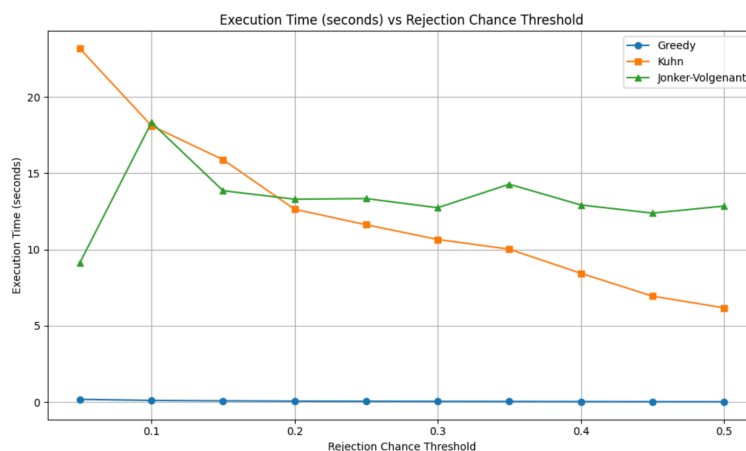


Figure 12: Effect of rejection threshold on runtime

Match count: As the rejection threshold increases, the graph becomes denser. Hence, more donor-recipient pairs satisfy the constraint, increasing matching size for all algorithms. Greedy and Kuhn’s algorithms once again show the sharpest increase, whereas for JV, the plateau is

once again a characteristic – confirming that the algorithm is driven by overall quality of matches, and not by cardinality.

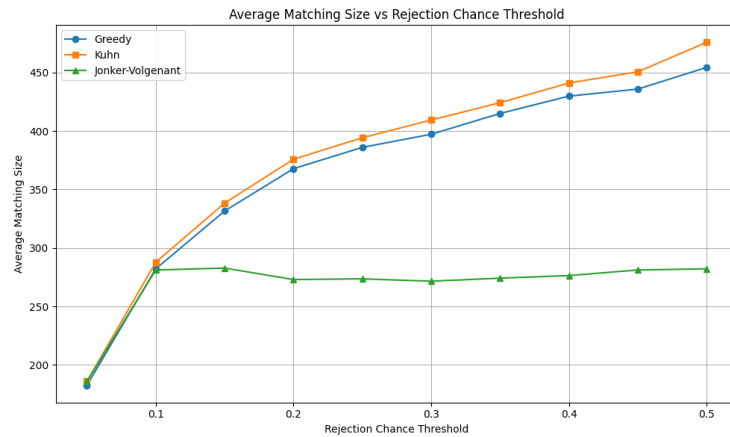


Figure 13: Effect of rejection threshold on matching size

Match quality: The quality metrics under varying thresholds reveal a dilution effect. At very low thresholds, only the most compatible pairs form edges. Hence, all three algorithms produce higher HLA and lower PRA values in their matches. As the threshold increases, lower quality pairs enter the graph, and the average HLA scores decline while average PRA rises across all algorithms. JV maintains, however, higher HLA scores than Greedy and Kuhn's throughout the experiment. The PRA results show that JV trends significantly higher than the other two algorithms at larger thresholds, and this is consistent with its behaviour – in larger thresholds, highly sensitized patients play an increasing role in minimizing the overall cost.

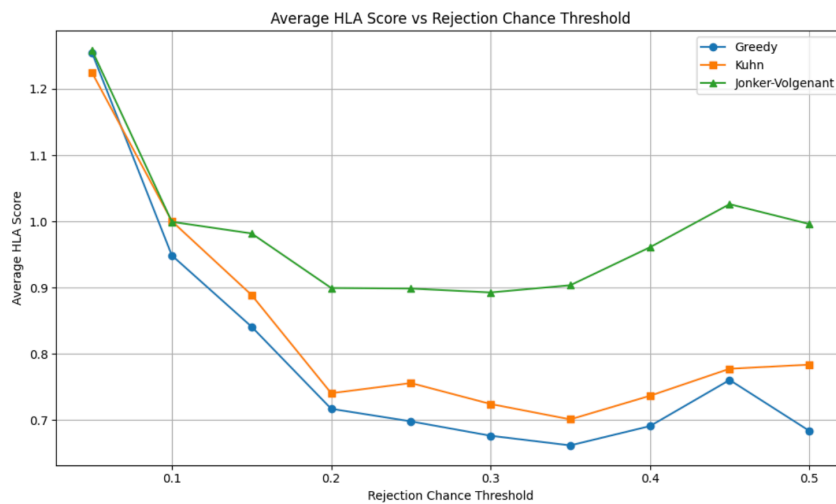


Figure 14: Effect of rejection threshold on HLA

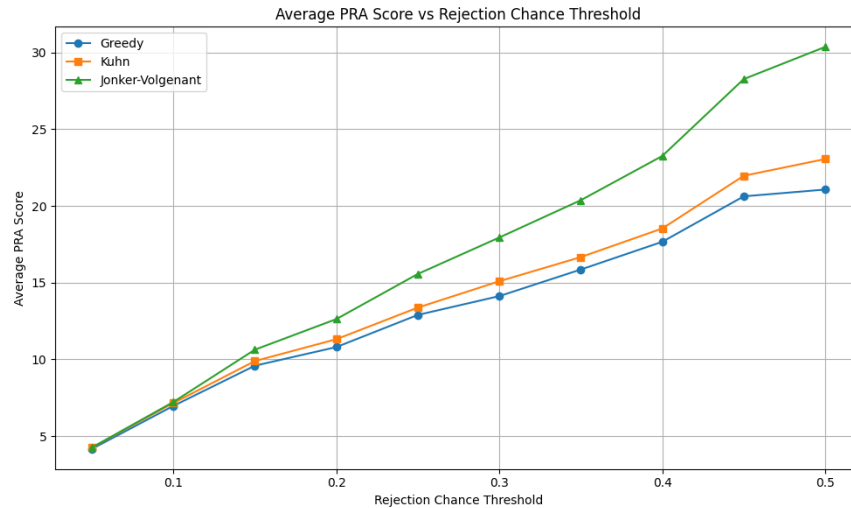


Figure 15: Effect of rejection threshold PRA

CONCLUSION

This study implemented three bipartite graph algorithms, Greedy, Kuhn's and Jonker-Volgenant, on randomly generated datasets for kidney compatibility between donors and recipients. The data was generated randomly, modeled after U.S./European populations. Results confirm that Greedy matching, while suboptimal, is the fastest by a significant margin. It consistently produces suboptimal outcomes in both quality and quantity of matches. Kuhn's algorithm, however, finds the maximum number of compatible matches through augmenting path search, outperforming Greedy in larger graphs. Jonker-Volgenant finds the same number of matches as Kuhn's at smaller graph sizes but diverges as n grows large – plateauing as it prioritizes quality. This produced consistently higher HLA scores and lower rejection risk across all three experiments. The rejection threshold experiment further confirms that algorithm choice has greatest impact at moderate graph densities, as at very low thresholds, all algorithms are bottlenecked by sparsity of graphs. At higher thresholds, the quality advantage of Jonker-Volgenant starts to be more pronounced.

The results carry implications for real-world organ allocation. The finding that all three algorithms converge at low donor-recipient ratios, which reflects the real-world scarcity of donors, suggests that algorithm choice has limited impact on the number of matches under real world constraints. However, Jonker-Volgenant's tendency to prioritize highly sensitized recipients with strong HLA matches at low donor ratios directly correlates with the goals of modern transplant policy, aiming to reduce risk of rejection but also the disadvantage faced by highly sensitized recipients on a transplant list. This suggests that cost-optimal weighted matching algorithms may offer some meaningful improvements over current heuristic allocation systems, not just in quantity but also regarding equity of access.

LIMITATIONS AND FUTURE WORK

It is important to acknowledge significant limitations in this work. While efforts were made to make the simulation as realistic as possible, time constraints and complexity of biological systems forced multiple simplifications in the work, along with limited number of experiments being conducted.

1. Datasets: The datasets used were synthetic, having only approximate distributions of attributes. This was especially the case with the HLA alleles, which in reality, have more complex distributions – this project also omitted the possibility of having homozygous alleles in a locus. Future work must conduct experiments with a large, varied dataset containing real-world samples.
2. Bipartite nature: In reality, kidney donor matching takes multiple variations such as: kidney swaps, donation chains, ABO-incompatible matching, etc. Donation chains, for example, are cyclical in nature. Hence, bipartite modelling may not be enough for these other variations. Future work must also examine such cases and analyse algorithms for their computational models.
3. Scalability: Given that the algorithms employed were computationally expensive, it was not feasible to test larger sample sizes to arrive at a broader conclusion. Future work may build on these results by employing larger sample sizes with more powerful computing resources.
4. Algorithms tested: Only three algorithms were tested in this approach, which leaves out the possibility of many other algorithms being more suitable for the problem. Future work may test the following possible algorithms:
 - a. Auction Algorithm (Parallelizable to reduce its cubic time complexity)
 - b. Hopcroft-Karp Algorithm
 - c. Ford-Fulkerson Algorithm
 - d. This work would also like to propose that future works develop novel algorithms that include heuristics to reward cardinality.
5. Limited attributes: Unlike the algorithm employed by UNOS, factors such as distance and urgency are not included. These attributes may have a more profound effect on how the algorithms behave in practice. Future work should consider these attributes along with medical constraints.

ACKNOWLEDGEMENTS

This study was conducted as part of coursework for the *Data Structures and Algorithms* course at *Rutgers – New Brunswick*. The author would like to thank *Dr. Shantenu Jha, PhD*, for his mentorship and tutelage throughout the semester. Furthermore, the author is grateful for the foundational knowledge in experiment design and scientific writing received during their undergraduate education at the *University of South Florida – Tampa*, which proved instrumental in the conception and execution of this study. This project would also not have been possible without *Dr. Oguzhan Topsakal, PhD*, whose teaching in the undergraduate course *COT4400 Analysis of Algorithms* at the *University of South Florida – Tampa*, and the opportunity for the author to serve as an Undergraduate Teaching Assistant, helped the author gain knowledge which were highly useful in this study.

REFERENCES

- [1] UNOS, "Organ Procurement and Transplantation Network (OPTN) Data," U.S. Department of Health & Human Services, 2023. [Online]. Available: <https://optn.transplant.hrsa.gov/data/>
- [2] University of Rochester Medical Center, "Organ Transplant Rejection," Life Sciences Learning Center, 2009. [Online]. Available: <https://www.urmc.rochester.edu/MediaLibraries/URMCMedia/life-sciences-learning-center/documents/STUDENTRejection7-23-09.pdf>
- [3] A. E. Roth, T. Sonmez, and M. U. Unver, "Kidney exchange," *Quarterly Journal of Economics*, vol. 119, no. 2, pp. 457–488, 2004.
- [4] C. Süsal and G. Opelz, "Current role of human leukocyte antigen matching in kidney transplantation," *Current Opinion in Organ Transplantation*, vol. 18, no. 4, pp. 438–444, 2013.
- [5] UNOS, "How we match organs," 2023. [Online]. Available: <https://unos.org/transplant/how-we-match-organs/>
- [6] S. A. Zenios, "Optimal control of a paired-kidney exchange program," *Management Science*, vol. 48, no. 3, pp. 328–342, 2002.
- [7] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [8] R. Jonker and A. Volgenant, "A shortest augmenting path algorithm for dense and sparse linear assignment problems," *Computing*, vol. 38, no. 4, pp. 325–340, 1987.
- [9] G. Garratty, S. A. Glynn, and R. McEntire, "ABO and Rh(D) phenotype frequencies of different racial/ethnic groups in the United States," *Transfusion*, vol. 44, no. 5, pp. 703–706, 2004.

DISCLOSURE OF AI ASSISTANCE

To uphold academic integrity and honesty, the use of Large Language Models (LLMs) is disclosed in this section. LLM models, including *Claude Sonnet* and *OpenAI ChatGPT*, were used during development to assist with:

- Presentation Refinement
- Refining and structuring the duality theory concept in the appendix for theoretical clarity
- Formatting and wording suggestions
- Finding biological distribution statistics
- Generation of pseudocode from implement algorithm
- Debugging algorithm implementation
- Conceptual clarification of the workings of the algorithms which were implemented
- Implementing *matplotlib* functions and re-designing driver program for ease of use
- Writing the README.md file for GitHub
- Understanding of content and pseudocode presented in resources

All LLM-generated outputs, including technical content, were verified independently by the author for correctness before being accepted into the work.

APPENDIX

Linear Programming and Duality for Jonker-Volgenant

Define:

- D = the set of donors
- R = the set of recipients
- $x(d, r) = \begin{cases} 1 & \text{if donor } d \text{ matched to recipient } r \\ 0 & \text{if no match between } d, r \end{cases}$
- $cost(d, r) = \frac{PRA(r)}{100} \cdot (1 - \frac{HLA\ score(d, r)}{100})$

Primal problem:

$$\min \sum_{d \in D} \sum_{r \in R} cost(d, r) \cdot x(d, r)$$

Subject to:

$$\sum_{r \in R} x(d, r) = 1 \quad \forall r \in R \text{ (every recipient matched only once)}$$

$$\sum_{d \in D} x(d, r) = 1 \quad \forall d \in D \text{ (every donor matched only once)}$$

Dual problem:

$$\max \sum_{d \in D} u[d] + \sum_{d \in D} v[r]$$

Subject to:

$$u(d) + v(r) \leq cost(d, r) \quad \forall d \in D, r \in R$$

where:

u represents dual constraint variables of each donor
 r represents dual constraint variables of each recipient

Complementary slackness

$rc(d, r) = cost(r, d) - u(d) - v(r)$ represents the reduced cost of edge(d, r)
 The dual constraint requires that $rc(d, r) \geq 0 \quad \forall d \in D, r \in R$

By complementary slackness, a primal solution and dual solution are both optimal if and only if
 for all matched pairs:

$$x(d, r) = 1 \Rightarrow u(d) + v(r) = cost(d, r) \quad \forall d \in D, r \in R$$

Since dual slackness forces:

$$x(d, r) \cdot (\text{cost}(d, r) - u(d) - v(r)) = 0 \quad \forall d \in D, r \in R,$$

we also know that $x(d, r) = 0$ for unmatched pairs.

This means that, for all matched pairs, $x(d, r) = 1$.
Hence, to uphold complementary slackness, $\text{cost}(d, r) - u(d) - v(r) = 0$ to be an active constraint. This property is then used to derive potential updates.

Potential updates

After each Dijkstra augmentation to free recipient r^* , the potentials must be updated to maintain complementary slackness for the new matching. Let $\text{dist}[r]$ represent the shortest path distance from current donor to a recipient r found by Dijkstra.

Rule 1: Update recipient potentials:

$$v[r] \leftarrow v[r] + \text{dist}[r] - \text{dist}[r^*] \quad \forall r \in R$$

Rule 2: Update matched donor potentials:

$$u[d] \leftarrow u[d] + \text{dist}[\text{matchD}[d]] - \text{dist}[r^*] \quad \forall d \text{ currently matched}$$

Rule 3: Update current donor potential:

$$u[\text{donor}] \leftarrow u[\text{donor}] + \text{dist}[r^*]$$

The distance array $\text{dist}[]$ is computed by Dijkstra's algorithm with reduced costs. Two cases arise during the search:

Rule 4: Initialize distances from the current donor d (no eviction):

$$\text{dist}[r] = \text{cost}(d, r) - u[d] - v[r] \quad \forall r \in R$$

Rule 5: Relax distances through an already-matched donor d_{old} (eviction case):

$$\text{dist}[r_2] = \text{dist}[r_1] + \text{cost}(d_{old}, r_2) - u[d_{old}] - v[r_2]$$

Where r_1 is the recipient currently matched to d_{old} , and r_2 is a candidate recipient for a new assignment. Rule 5 is simply the standard Dijkstra's relaxation step applied to reduced costs. Through an existing match, a shortest path is found. This corresponds to an

augmenting path that reassigns d_{old} from r_1 to r_2 .